

58. Which integers are quadratic residues of 11?
59. Show that if p is an odd prime and a is an integer not divisible by p , then the congruence $x^2 \equiv a \pmod{p}$ has either no solutions or exactly two incongruent solutions modulo p .

60. Show that if p is an odd prime, then there are exactly $(p-1)/2$ quadratic residues of p among the integers $1, 2, \dots, p-1$.

If p is an odd prime and a is an integer not divisible by p , the **Legendre symbol** $\left(\frac{a}{p}\right)$ is defined to be 1 if a is a quadratic residue of p and -1 otherwise.

61. Show that if p is an odd prime and a and b are integers with $a \equiv b \pmod{p}$, then

$$\left(\frac{a}{p}\right) = \left(\frac{b}{p}\right).$$

62. Prove **Euler's criterion**, which states that if p is an odd prime and a is a positive integer not divisible by p , then

$$\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}.$$

[Hint: If a is a quadratic residue modulo p , apply Fermat's little theorem; otherwise, apply Wilson's theorem, given in Exercise 18(b).]

63. Use Exercise 62 to show that if p is an odd prime and a and b are integers not divisible by p , then

$$\left(\frac{ab}{p}\right) = \left(\frac{a}{p}\right)\left(\frac{b}{p}\right).$$

64. Show that if p is an odd prime, then -1 is a quadratic residue of p if $p \equiv 1 \pmod{4}$, and -1 is not a quadratic residue of p if $p \equiv 3 \pmod{4}$. [Hint: Use Exercise 62.]
65. Find all solutions of the congruence $x^2 \equiv 29 \pmod{35}$. [Hint: Find the solutions of this congruence modulo 5 and modulo 7, and then use the Chinese remainder theorem.]
66. Find all solutions of the congruence $x^2 \equiv 16 \pmod{105}$. [Hint: Find the solutions of this congruence modulo 3, modulo 5, and modulo 7, and then use the Chinese remainder theorem.]
67. Describe a brute-force algorithm for solving the discrete logarithm problem and find the worst-case and average-case time complexity of this algorithm.

4.5 Applications of Congruences

Congruences have many applications to discrete mathematics, computer science, and many other disciplines. We will introduce three applications in this section: the use of congruences to assign memory locations to computer files, the generation of pseudorandom numbers, and check digits.

Suppose that a customer identification number is ten digits long. To retrieve customer files quickly, we do not want to assign a memory location to a customer record using the ten-digit identification number. Instead, we want to use a smaller integer associated to the identification number. This can be done using what is known as a hashing function. In this section we will show how we can use modular arithmetic to do hashing.

Constructing sequences of random numbers is important for randomized algorithms, for simulations, and for many other purposes. Constructing a sequence of truly random numbers is extremely difficult, or perhaps impossible, because any method for generating what are supposed to be random numbers may generate numbers with hidden patterns. As a consequence, methods have been developed for finding sequences of numbers that have many desirable properties of random numbers, and which can be used for various purposes in place of random numbers. In this section we will show how to use congruences to generate sequences of pseudorandom numbers. The advantage is that the pseudorandom numbers so generated are constructed quickly; the disadvantage is that they have too much predictability to be used for many tasks.

Congruences also can be used to produce check digits for identification numbers of various kinds, such as code numbers used to identify retail products, numbers used to identify books, airline ticket numbers, and so on. We will explain how to construct check digits using congruences for a variety of types of identification numbers. We will show that these check digits can be used to detect certain kinds of common errors made when identification numbers are printed.

4.5.1 Hashing Functions



The central computer at an insurance company maintains records for each of its customers. How can memory locations be assigned so that customer records can be retrieved quickly? The solution to this problem is to use a suitably chosen **hashing function**. Records are identified using a **key**, which uniquely identifies each customer's records. For instance, customer records are often identified using the Social Security number of the customer as the key. A hashing function h assigns memory location $h(k)$ to the record that has k as its key.

In practice, many different hashing functions are used. One of the most common is the function

$$h(k) = k \bmod m$$

where m is the number of available memory locations.

Hashing functions should be easily evaluated so that files can be quickly located. The hashing function $h(k) = k \bmod m$ meets this requirement; to find $h(k)$, we need only compute the remainder when k is divided by m . Furthermore, the hashing function should be onto, so that all memory locations are possible. The function $h(k) = k \bmod m$ also satisfies this property.

EXAMPLE 1 Find the memory locations assigned by the hashing function $h(k) = k \bmod 111$ to the records of customers with Social Security numbers 064212848 and 037149212.

Solution: The record of the customer with Social Security number 064212848 is assigned to memory location 14, because

$$h(064212848) = 064212848 \bmod 111 = 14.$$

Similarly, because

$$h(037149212) = 037149212 \bmod 111 = 65,$$

the record of the customer with Social Security number 037149212 is assigned to memory location 65. 

Because a hashing function is not one-to-one (because there are more possible keys than memory locations), more than one file may be assigned to a memory location. When this happens, we say that a **collision** occurs. One way to resolve a collision is to assign the first free location following the occupied memory location assigned by the hashing function.

EXAMPLE 2 After making the assignments of records to memory locations in Example 1, assign a memory location to the record of the customer with Social Security number 107405723.

Solution: First note that the hashing function $h(k) = k \bmod 111$ maps the Social Security number 107405723 to location 14, because

$$h(107405723) = 107405723 \bmod 111 = 14.$$

However, this location is already occupied (by the file of the customer with Social Security number 064212848). But, because memory location 15, the first location following memory location 14, is free, we assign the record of the customer with Social Security number 107405723 to this location. 

In Example 1 we used a **linear probing function**, namely, $h(k, i) = h(k) + i \bmod m$, to look for the first free memory location, where i runs from 0 to $m - 1$. There are many other ways to resolve collisions that are discussed in the references on hashing functions given at the end of the book.

4.5.2 Pseudorandom Numbers

Randomly chosen numbers are often needed for computer simulations. Different methods have been devised for generating numbers that have properties of randomly chosen numbers. Because numbers generated by systematic methods are not truly random, they are called **pseudorandom numbers**.

Links ▶ The most commonly used procedure for generating pseudorandom numbers is the **linear congruential method**. We choose four integers: the **modulus** m , **multiplier** a , **increment** c , and **seed** x_0 , with $2 \leq a < m$, $0 \leq c < m$, and $0 \leq x_0 < m$. We generate a sequence of pseudorandom numbers $\{x_n\}$, with $0 \leq x_n < m$ for all n , by successively using the recursively defined function

$$x_{n+1} = (ax_n + c) \bmod m.$$

(This is an example of a recursive definition, discussed in Section 5.3. In that section we will show that such sequences are well defined.)

Many computer experiments require the generation of pseudorandom numbers between 0 and 1. To generate such numbers, we divide numbers generated with a linear congruential generator by the modulus: that is, we use the numbers x_n/m .

EXAMPLE 3 Find the sequence of pseudorandom numbers generated by the linear congruential method with modulus $m = 9$, multiplier $a = 7$, increment $c = 4$, and seed $x_0 = 3$.

Solution: We compute the terms of this sequence by successively using the recursively defined function $x_{n+1} = (7x_n + 4) \bmod 9$, beginning by inserting the seed $x_0 = 3$ to find x_1 . We find that

$$\begin{aligned} x_1 &= 7x_0 + 4 \bmod 9 = 7 \cdot 3 + 4 \bmod 9 = 25 \bmod 9 = 7, \\ x_2 &= 7x_1 + 4 \bmod 9 = 7 \cdot 7 + 4 \bmod 9 = 53 \bmod 9 = 8, \\ x_3 &= 7x_2 + 4 \bmod 9 = 7 \cdot 8 + 4 \bmod 9 = 60 \bmod 9 = 6, \\ x_4 &= 7x_3 + 4 \bmod 9 = 7 \cdot 6 + 4 \bmod 9 = 46 \bmod 9 = 1, \\ x_5 &= 7x_4 + 4 \bmod 9 = 7 \cdot 1 + 4 \bmod 9 = 11 \bmod 9 = 2, \\ x_6 &= 7x_5 + 4 \bmod 9 = 7 \cdot 2 + 4 \bmod 9 = 18 \bmod 9 = 0, \\ x_7 &= 7x_6 + 4 \bmod 9 = 7 \cdot 0 + 4 \bmod 9 = 4 \bmod 9 = 4, \\ x_8 &= 7x_7 + 4 \bmod 9 = 7 \cdot 4 + 4 \bmod 9 = 32 \bmod 9 = 5, \\ x_9 &= 7x_8 + 4 \bmod 9 = 7 \cdot 5 + 4 \bmod 9 = 39 \bmod 9 = 3. \end{aligned}$$

Because $x_9 = x_0$ and because each term depends only on the previous term, we see that the sequence

$$3, 7, 8, 6, 1, 2, 0, 4, 5, 3, 7, 8, 6, 1, 2, 0, 4, 5, 3, \dots$$

is generated. This sequence contains nine different numbers before repeating. ◀

Most computers do use linear congruential generators to generate pseudorandom numbers. Often, a linear congruential generator with increment $c = 0$ is used. Such a generator is called

a **pure multiplicative generator**. For example, the pure multiplicative generator with modulus $2^{31} - 1$ and multiplier $7^5 = 16,807$ is widely used. With these values, it can be shown that $2^{31} - 2$ numbers are generated before repetition begins.

Pseudorandom numbers generated by linear congruential generators have long been used for many tasks. Unfortunately, it has been shown that sequences of pseudorandom numbers generated in this way do not share some important statistical properties that true random numbers have. Because of this, it is not advisable to use them for some tasks, such as large simulations. For such sensitive tasks, other methods are used to produce sequences of pseudorandom numbers, either using some sort of algorithm or sampling numbers arising from a random physical phenomenon. For more details on pseudorandom number, see [Kn97] and [Re10].

4.5.3 Check Digits

Congruences are used to check for errors in digit strings. A common technique for detecting errors in such strings is to add an extra digit at the end of the string. This final digit, or check digit, is calculated using a particular function. Then, to determine whether a digit string is correct, a check is made to see whether this final digit has the correct value. We begin with an application of this idea for checking the correctness of bit strings.

EXAMPLE 4 Parity Check Bits Digital information is represented by bit string, split into blocks of a specified size. Before each block is stored or transmitted, an extra bit, called a **parity check bit**, can be appended to each block. The parity check bit x_{n+1} for the bit string $x_1 x_2 \dots x_n$ is defined by

$$x_{n+1} = x_1 + x_2 + \dots + x_n \bmod 2.$$

It follows that x_{n+1} is 0 if there are an even number of 1 bits in the block of n bits and it is 1 if there are an odd number of 1 bits in the block of n bits. When we examine a string that includes a parity check bit, we know that there is an error in it if the parity check bit is wrong. However, when the parity check bit is correct, there still may be an error. A parity check can detect an odd number of errors in the previous bits, but not an even number of errors. (See Exercise 14.)

Suppose we receive in a transmission the bit strings 01100101 and 11010110, each ending with a parity check bit. Should we accept these bit strings as correct?

Solution: Before accepting these strings as correct, we examine their parity check bits. The parity check bit of the first string is 1. Because $0 + 1 + 1 + 0 + 0 + 1 + 0 \equiv 1 \pmod{2}$, the parity check bit is correct. The parity check bit of the second string is 0. We find that $1 + 1 + 0 + 1 + 0 + 1 + 1 \equiv 1 \pmod{2}$, so the parity check is incorrect. We conclude that the first string may have been transmitted correctly and we know for certain that the second string was transmitted incorrectly. We accept the first string as correct (even though it still may contain an even number of errors), but we reject the second string. 

Check bits computed using congruences are used extensively to verify the correctness of various kinds of identification numbers. Examples 5 and 6 show how check bits are computed for codes that identify products (Universal Product Codes) and books (International Standard Book Numbers). The preambles to Exercises 18, 28, and 32 introduce the use of congruences to find and use check digits in money order numbers, airline ticket numbers, and identification numbers for periodicals, respectively. Note that congruences are also used to compute check digits for bank account numbers, drivers license numbers, credit card numbers, and many other types of identification numbers.

EXAMPLE 5 UPCs Retail products are identified by their **Universal Product Codes (UPCs)**. The most common form of a UPC has 12 decimal digits: the first digit identifies the product category, the

next five digits identify the manufacturer, the following five identify the particular product, and the last digit is a check digit. The check digit is determined by the congruence

$$3x_1 + x_2 + 3x_3 + x_4 + 3x_5 + x_6 + 3x_7 + x_8 + 3x_9 + x_{10} + 3x_{11} + x_{12} \equiv 0 \pmod{10}.$$

Answer these questions:

- (a) Suppose that the first 11 digits of a UPC are 79357343104. What is the check digit?
 (b) Is 041331021641 a valid UPC?

Solution: (a) We insert the digits of 79357343104 into the congruence for UPC check digits. This gives $3 \cdot 7 + 9 + 3 \cdot 3 + 5 + 3 \cdot 7 + 3 + 3 \cdot 4 + 3 + 3 \cdot 1 + 0 + 3 \cdot 4 + x_{12} \equiv 0 \pmod{10}$. Simplifying, we have $21 + 9 + 9 + 5 + 21 + 3 + 12 + 3 + 3 + 0 + 12 + x_{12} \equiv 0 \pmod{10}$. Hence, $98 + x_{12} \equiv 0 \pmod{10}$. It follows that $x_{12} \equiv 2 \pmod{10}$, so the check digit is 2.

(b) To check whether 041331021641 is valid, we insert the digits into the congruence these digits must satisfy. This gives $3 \cdot 0 + 4 + 3 \cdot 1 + 3 + 3 \cdot 3 + 1 + 3 \cdot 0 + 2 + 3 \cdot 1 + 6 + 3 \cdot 4 + 1 \equiv 0 + 4 + 3 + 3 + 9 + 1 + 0 + 2 + 3 + 6 + 12 + 1 \equiv 4 \not\equiv 0 \pmod{10}$. Hence, 041331021641 is not a valid UPC. 

EXAMPLE 6

Remember that the check digit of an ISBN-10 can be an X!

ISBNs All books are identified by an **International Standard Book Number (ISBN-10)**, a 10-digit code $x_1x_2 \dots x_{10}$, assigned by the publisher. (Recently, a 13-digit code known as ISBN-13 was introduced to identify a larger number of published works; see the preamble to Exercise 42 in the Supplementary Exercises.) An ISBN-10 consists of blocks identifying the language, the publisher, the number assigned to the book by its publishing company, and finally, a check digit that is either a digit or the letter X (used to represent 10). This check digit is selected so that

$$x_{10} \equiv \sum_{i=1}^9 ix_i \pmod{11},$$

or equivalently, so that

$$\sum_{i=1}^{10} ix_i \equiv 0 \pmod{11}.$$

Answer these questions about ISBN-10s:

- (a) The first nine digits of the ISBN-10 of the sixth edition of this book are 007288008. What is the check digit?
 (b) Is 084930149X a valid ISBN-10?

Solution: (a) The check digit is determined by the congruence $\sum_{i=1}^{10} ix_i \equiv 0 \pmod{11}$. Inserting the digits 007288008 gives $x_{10} \equiv 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 7 + 4 \cdot 2 + 5 \cdot 8 + 6 \cdot 8 + 7 \cdot 0 + 8 \cdot 0 + 9 \cdot 8 \pmod{11}$. This means that $x_{10} \equiv 0 + 0 + 21 + 8 + 40 + 48 + 0 + 0 + 72 \pmod{11}$, so $x_{10} \equiv 189 \equiv 2 \pmod{11}$. Hence, $x_{10} = 2$.

(b) To see whether 084930149X is a valid ISBN-10, we see if $\sum_{i=1}^{10} ix_i \equiv 0 \pmod{11}$. We see that $1 \cdot 0 + 2 \cdot 8 + 3 \cdot 4 + 4 \cdot 9 + 5 \cdot 3 + 6 \cdot 0 + 7 \cdot 1 + 8 \cdot 4 + 9 \cdot 9 + 10 \cdot 10 = 0 + 16 + 12 + 36 + 15 + 0 + 7 + 32 + 81 + 100 = 299 \equiv 2 \not\equiv 0 \pmod{11}$. Hence, 084930149X is not a valid ISBN-10. 

Publishers sometimes do not calculate ISBNs correctly for their books, as was done for an earlier edition of this text.

Several kinds of errors often arise in identification numbers. A **single error**, an error in one digit of an identification number, is perhaps the most common type of error. Another common kind of error is a **transposition error**, which occurs when two digits are accidentally interchanged. For each type of identification number, including a check digit, we would like to be able to detect these common types of errors, as well as other types of errors. We will investigate whether the check digit for ISBNs can detect single errors and transposition errors. Whether check digits for UPCs can detect these kinds of errors is left as Exercises 26 and 27.

Suppose that $x_1x_2 \dots x_{10}$ is a valid ISBN (so that $\sum_{i=1}^{10} x_i \equiv 0 \pmod{10}$). We will show that we can detect a single error and a transposition of two digits (where we include the possibility that one of the two digits is the check digit X, representing 10). Suppose that this ISBN has been printed with a single error as $y_1y_2 \dots y_{10}$. If there is a single error, then, for some integer j , $y_i = x_i$ for $i \neq j$ and $y_j = x_j + a$, where $-10 \leq a \leq 10$ and $a \neq 0$. Note that $a = y_j - x_j$ is the error in the j th place. It then follows that

$$\sum_{i=1}^{10} iy_i = \left(\sum_{i=1}^{10} ix_i \right) + ja \equiv ja \not\equiv 0 \pmod{11}.$$

These last two congruences hold because $\sum_{i=1}^{10} x_i \equiv 0 \pmod{10}$ and $11 \nmid ja$, because $11 \nmid j$ and $11 \nmid a$. We conclude that $y_1y_2 \dots y_{10}$ is not a valid ISBN. So, we have detected the single error.

Now suppose that two unequal digits have been transposed. It follows that there are distinct integers j and k such that $y_j = x_k$ and $y_k = x_j$, and $y_i = x_i$ for $i \neq j$ and $i \neq k$. Hence,

$$\sum_{i=1}^{10} iy_i = \left(\sum_{i=1}^{10} ix_i \right) + (jx_k - jx_j) + (kx_j - kx_k) \equiv (j - k)(x_k - x_j) \not\equiv 0 \pmod{11},$$

because $\sum_{i=1}^{10} x_i \equiv 0 \pmod{10}$ and $11 \nmid (j - k)$ and $11 \nmid (x_k - x_j)$. We see that $y_1y_2 \dots y_{10}$ is not a valid ISBN. Thus, we can detect the interchange of two unequal digits.

Exercises

- Which memory locations are assigned by the hashing function $h(k) = k \bmod 97$ to the records of insurance company customers with these Social Security numbers?
 - 034567981
 - 183211232
 - 220195744
 - 987255335
- Which memory locations are assigned by the hashing function $h(k) = k \bmod 101$ to the records of insurance company customers with these Social Security numbers?
 - 104578690
 - 432222187
 - 372201919
 - 501338753
- A parking lot has 31 visitor spaces, numbered from 0 to 30. Visitors are assigned parking spaces using the hashing function $h(k) = k \bmod 31$, where k is the number formed from the first three digits on a visitor's license plate.
 - Which spaces are assigned by the hashing function to cars that have these first three digits on their license plates: 317, 918, 007, 100, 111, 310?
 - Describe a procedure visitors should follow to find a free parking space, when the space they are assigned is occupied.

Another way to resolve collisions in hashing is to use *double hashing*. We use an initial hashing function $h(k) = k \bmod p$, where p is prime. We also use a second hashing function $g(k) = (k + 1) \bmod (p - 2)$. When a collision occurs, we use a *probing sequence* $h(k, i) = (h(k) + i \cdot g(k)) \bmod p$.

- Use the double hashing procedure we have described with $p = 4969$ to assign memory locations to files for employees with social security numbers $k_1 = 132489971$, $k_2 = 509496993$, $k_3 = 546332190$, $k_4 = 034367980$, $k_5 = 047900151$, $k_6 = 329938157$, $k_7 = 212228844$, $k_8 = 325510778$, $k_9 = 353354519$, $k_{10} = 053708912$.
- What sequence of pseudorandom numbers is generated using the linear congruential generator $x_{n+1} = (3x_n + 2) \bmod 13$ with seed $x_0 = 1$?
- What sequence of pseudorandom numbers is generated using the linear congruential generator $x_{n+1} = (4x_n + 1) \bmod 7$ with seed $x_0 = 3$?
- What sequence of pseudorandom numbers is generated using the pure multiplicative generator $x_{n+1} = 3x_n \bmod 11$ with seed $x_0 = 2$?

8. Write an algorithm in pseudocode for generating a sequence of pseudorandom numbers using a linear congruential generator.

The **middle-square method** for generating pseudorandom numbers begins with an n -digit integer. This number is squared, initial zeros are appended to ensure that the result has $2n$ digits, and its middle n digits are used to form the next number in the sequence. This process is repeated to generate additional terms.

9. Find the first eight terms of the sequence of four-digit pseudorandom numbers generated by the middle square method starting with 2357.
10. Explain why both 3792 and 2916 would be bad choices for the initial term of a sequence of four-digit pseudorandom numbers generated by the middle square method.
- The **power generator** is a method for generating pseudorandom numbers. To use the power generator, parameters p and d are specified, where p is a prime, d is a positive integer such that $p \nmid d$, and a seed x_0 is specified. The pseudorandom numbers $x_1, x_2, \dots$ are generated using the recursive definition $x_{n+1} = x_n^d \bmod p$.
11. Find the sequence of pseudorandom numbers generated by the power generator with $p = 7$, $d = 3$, and seed $x_0 = 2$.
12. Find the sequence of pseudorandom numbers generated by the power generator with $p = 11$, $d = 2$, and seed $x_0 = 3$.
13. Suppose you received these bit strings over a communications link, where the last bit is a parity check bit. In which string are you sure there is an error?
- a) 00000111111 b) 10101010101
c) 11111100000 d) 10111101111
14. Prove that a parity check bit can detect an error in a string if and only if the string contains an odd number of errors.

15. The first nine digits of the ISBN-10 of the European version of the fifth edition of this book are 0-07-119881. What is the check digit for that book?
16. The ISBN-10 of the sixth edition of *Elementary Number Theory and Its Applications* is 0-321-500Q1-8, where Q is a digit. Find the value of Q .
17. Determine whether the check digit of the ISBN-10 for this textbook (the eighth edition of *Discrete Mathematics and Its Applications*) was computed correctly by the publisher.

The United States Postal Service (USPS) sells money orders identified by an 11-digit number $x_1x_2 \dots x_{11}$. The first ten digits identify the money order; x_{11} is a check digit that satisfies $x_{11} = x_1 + x_2 + \dots + x_{10} \bmod 9$.

18. Find the check digit for the USPS money orders that have identification number that start with these ten digits.
- a) 7555618873 b) 6966133421
c) 8018927435 d) 3289744134
19. Determine whether each of these numbers is a valid USPS money order identification number.
- a) 74051489623 b) 88382013445
c) 56152240784 d) 66606631178
20. One digit in each of these identification numbers of a postal money order is smudged. Can you recover the smudged digit, indicated by a Q , in each of these numbers?
- a) $Q1223139784$ b) $6702120Q988$
c) $27Q41007734$ d) $213279032Q1$
21. One digit in each of these identification numbers of a postal money order is smudged. Can you recover the smudged digit, indicated by a Q , in each of these numbers?
- a) $493212Q0688$ b) $850Q9103858$
c) $2Q941007734$ d) $66687Q03201$
22. Determine which single digit errors are detected by the USPS money order code.
23. Determine which transposition errors are detected by the USPS money order code.
24. Determine the check digit for the UPCs that have these initial 11 digits.
- a) 73232184434 b) 63623991346
c) 04587320720 d) 93764323341
25. Determine whether each of the strings of 12 digits is a valid UPC code.
- a) 036000291452 b) 012345678903
c) 782421843014 d) 726412175425
26. Does the check digit of a UPC code detect all single errors? Prove your answer or find a counterexample.
27. Determine which transposition errors the check digit of a UPC code finds.

Some airline tickets have a 15-digit identification number $a_1a_2 \dots a_{15}$, where a_{15} is a check digit that equals $a_1a_2 \dots a_{14} \bmod 7$.

28. Find the check digit a_{15} that follows each of these initial 14 digits of an airline ticket identification number.
- a) 10237424413392 b) 00032781811234
c) 00611232134231 d) 00193222543435
29. Determine whether each of these 15-digit numbers is a valid airline ticket identification number.
- a) 101333341789013 b) 007862342770445
c) 113273438882531 d) 000122347322871
30. Which errors in a single digit of a 15-digit airline ticket identification number can be detected?
- *31. Can the accidental transposition of two consecutive digits in an airline ticket identification number be detected using the check digit?